SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA14 COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool. *(BL3)*

Assessment Tool Weightage: 40%

QUESTIONS: 5 Total Marks:50

Mode: In Class Hours : 60 mts

Annexure A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

I J. Bhanuprakash Reddy /192521171 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: J. Bhanuprakash Reddy

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	/ 20	/ 25	/ 20	/ 15	/ 100

Annexure A

ANALYTICAL PROBLEM SOLVING

1	Trace the processing of the input expression P = (a * b) + (b * c) * 2 through all phases of a compiler. a) Identify and list all tokens generated during lexical analysis. b) Construct the syntax tree during the parsing phase. c) Generate the intermediate code for the given expression. d) Apply suitable optimizations to the intermediate code. e) Explain the significance of the final target code prod
2	For each of the following Regular Expressions. construct the language. Write any FIVE valid strings for each Regular Expression. a)$(\varepsilon+aa^*)^*$ b)$(a^*+b^*)^*\ abb$ c)$(a^*b^*)^*\ aba\ (a^*+b^*)^*$ d)$(a+ab)^*$ e) $(aba)^*+(a^*b^*)^*$

3	i)Find the number of tokens in the following C statement is
	main( )
	{
	int a=10;
	char b =”abc”;
	in t c = 30;
	ch ar d = “xyz”;
	in /* comment */ t m = 40.5;
	}
	ii)Identify the lexical errors in the following C statement is
	void main()
	{
	int x=10, y=20;
	char * a;
	a= &x;
	x= 1xab;
	}

4	**Find a regular expression corresponding to each of the following subsets of $\{0,1\}^*$:** i) The language of all strings that have an even number of 0’s. ii) The language of all strings that contain at least one 1. iii) The language of all strings in which both the number of 0’s and the number of 1’s are odd. iv) The language of all strings that start with 1 and end with 0. v) The language of all strings that do not contain consecutive 1’s.

5	**Consider a lexical analyzer for a simplified programming language. The language consists of the following tokens:** 1. Keywords: if, else, while, return, int, float 2. Operators: +, -, *, /, =, ==, <, > 3. Separators: (,), {, }, ;, , 4. Identifiers: A sequence of letters followed by a sequence of digits, e.g., var123 5. Integer Literals: A sequence of digits, e.g., 12345 6. Floating-point Literals: A sequence of digits followed by a dot and another sequence of digits, e.g., 123.45 **The lexical analyzer needs to tokenize the following input string:** int var123 = 12345; float var456 = 123.45; if (var123 == var456) { return; } **Questions:** a) How many tokens are there in the given input string? b) Provide a list of tokens categorized by their types (keywords, operators, separators, identifiers, integer literals, floating-point literals). c) Assuming a finite state machine (FSM) is used for lexical analysis, estimate the number of state transitions required to tokenize the given input string. Provide your reasoning based on the transitions needed for each token type.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method

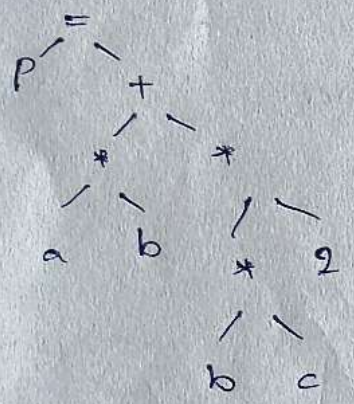
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

1. Phases of a compiler for $P = (a*b) + (b*c)*2$

a. Lexical Analysis :

Lexeme	Token Type
P, a, b, b, c	Identifier (5)
=	Assignment Operator
*, +, *, *	Arithmetic Operators (4)
(,), (,)	Paranthese (4)
2	Integer Constant

b. Syntax Tree :



90/100

1/3/0/26

c. Intermediate Code :

$t1 = a*b$

$t2 = b*c$

$t3 = t2*2$

$t4 = t1 + t3$

$P = t4$

d. Optimization

Copy propagation removes the temporary t4, so the last step becomes P = t1 + t3

Strength reduction replaces the multiplication by 2 with a shift (t2 << 1) or an addition (t2 + t2), both of which are cheaper than a multiply.

```
t1 = a * b
t2 = b * c
t3 = t2 << 1
P  = t1 + t3
```

This is 4 instructions instead of 5, and one multiplication is gone.

e. Target Code and its Significance.

```
LD   R1, a
LD   R2, b
MUL  R1, R1, R2    ; a*b
LD   R3, c
MUL  R2, R2, R3    ; b*c
SHL  R2, R2, 1     ; (b*c)*2
ADD  R1, R1, R2
ST   P , R1
```

The target code is the machine-level form that the hardware actually executes. Instruction selection and register allocation happen here.

2. Construction of Languages from Regular Expression.

Given: The alphabet is $\Sigma = \{a, b\}$

a. $(\varepsilon + aa^*)^*$

Construction of language:

aa^* represents one or more occurrences of a.

$\varepsilon + aa^*$ represents either the empty string or one or more a's.

Therefore, the language is:

$$L = \{\varepsilon, a, aa, aaa, \ldots\}$$

Any five valid strings:

1. a
2. aa
3. aaa
4. aaaa
5. aaaaa

b. $(a^* + b^*)^* abb$

a^* represents zero or more a's.

b^* represents zero or more b's.

$(a^* + b^*)^*$ allows any combination of a's and b's.

$$L = \{xabb \mid x \in \{a, b\}^*\}$$

Any five valid strings.

1. abb
2. aabb
3. babb
4. ababb
5. bbabb

c. $(a^+ b^*)^* aba(a^*+b^*)^*$

$(a^*b^*)^*$ can generate any string over $\{a,b\}$

$(a^+ + b^*)^*$ can generate any string over $\{a,b\}$ after aba.

$L = \{xabay \mid x,y \in \{a,b\}^*\}$

Any five valid strings

1. aba
2. aaba
3. abaa
4. baba
5. ababa

d. $(a+ab)^*$

a+ab represents either a or ab.

$L = \{a, ab\}^*$

Any five valid strings

1.	a	a
2.	ab	ab
3.	aa	a+a
4.	aab	a+ab
5.	abab	ab+ab

e. $(ab)^* + (a^*b^*)^*$

$(ab)^*$ represents zero or more repetitions of ab.

$(a^*b^*)^*$ can generate every possible string over $\{a,b\}$.

$L = \{a,b\}^*$

Any five valid strings:

1. a
2. b
3. ab
4. ba
5. aa

3. Lexical Analysis in C programming.

Given program:

```
main()
{
    int a=10;
    char b="abc";
    int c=30;
    char d="xyz";
    in /* comment */tm=40.5;
}
```

Token-by-Token Analysis

S.No	Token	Type
1	main	identifier
2	(	Punctuator
3	)	Punctuator
4	{	Punctuator
5	int	Keyword
6	a	identifier
7	=	Operator
8	10	Integer
9	;	Punctuator
10	char	Keyword
11	b	identifier
12	=	Operator
13	"abc"	String
14	;	Punctuator
15	in	identifier
16	t	identifier
17	c	identifier
18	=	Operator
19	30	Integer

20	;	Punctuator
21	ch	identifier
22	ar	identifier
23	d	identifier
24	=	Operator
25	"xyz"	String
26	;	Punctuator
27	in	identifier
28	t	identifier
29	m	identifier
30	=	Operator
31	40.5	Floating constant
32	;	Punctuator
33	}	Punctuator

b. Identification of errors

S.No	Statement	Error / Observation
1	Void main( )	No lexical error. void main( ) is non-standard in hosted c
2	int x=10, y=20;	No lexical error, Valid declaration and initialization
3	char *a;	No lexical error. Valid pointer declaration
4	a = &x;	No lexical error. It has a semantic type mismatch.
5	x = 1xab	Lexical error: 1xab is not a valid integer constant.

4. Regular Expressions for Given Languages

Given : $\Sigma = \{0, 1\}$

i. The language of all strings that have an even number of 0s.

Regular Expression : $1^*(01^*01^*)^*$

1^* represents any number of 1s.

01^*01^* represents two 0s with any number of 1s between and after them.

Examples:

String	Number of 0s
ε	0
1	0
00	2

ii. The language of all strings that contain atleast one 1.

Regular Expression: $(0+1)^*1(0+1)^*$

Explanation:

$(0+1)^*$ represents any combination of 0s and 1s.

The middle 1 ensures that every string contains atleast one 1.

Examples:

String → 1, 01, 10, 101, 0011

iii. The language of all strings in which both the number of 0s and the numbers of 1s are odd.

Both the number of 0s and the number of 1s must be odd.

$E = (00+11)$, which contributes an even number.

$0 = (01+10)$, which contributes one 0 and one 1.

Regular Expression:

$(00+11)^*(01+10)$

$((00+11)^*(01+10)(00+11)^*(01+10))^*(00+11)^*$

Example:

String	Number of 0s	Number of 1s
01	1	1
10	1	1
0011	2	2

v. The language of all strings that do not contain consecutive 1s.

0 represents a single zero.

$(0+10)^*$ allows any number of these blocks.

$(\varepsilon+1)$ allows an optional 1 at the end.

Regular Expression:

$(0+10)^*(\varepsilon+1)$

Examples

S.No	String
1	ε
2	0
3	1
4	1010
5	01010

5. Design of a Lexical Analyzer for a Simplified programming Language.

Given: A simplified programming language consisting of keywords, operators, separators, identifiers, integer and floating-point literals.

Input String:

```
int var123 = 12345;
float var456 = 123.45;
if (var123 == var456) {
    return;
}
```

a. How many tokens are there in the given input string:

S.No	Token	Token Type
1.	int, float, if return	Keyword
2.	var123, var456 Var123, var456	Identifier
3.	=, =, ==	Operator
4.	;, ;, (,), {, ;, }	Separator
5.	12345	Integer Literal
6.	123.45	Floating-point literal

Final Answer: 20 tokens

b. Provide a list of tokens categorized by their types

The tokens are classified into the following six categories.

S No	Token Type	Tokens	Count
1.	Keywords	int, float, if, return	4
2.	Operators	=, =, ==	3
3.	Separators	;, ;, (,), {, ;, }	7
4.	Identifiers	var123, var456, var123, var456	4
5.	Integer literals	12345	1
6.	Floating-point literals	123.45	1

c. Estimate the number of state transitions required to tokenize the given input string.

A finite state Machine (FSM) is a mathematical model used by a lexical analyzer to recognize tokens by moving between states according to the input characters.

whitespace Transitions = 14

T = Token transitions + Whitespace transitions

T = 62 + 14

T = 76

76 transitions